



# High-fidelity Endoscopic Image Synthesis via 3D Gaussian Splatting and Diffusion Implicit Model Integration

Qinyan Gong<sup>1</sup>

MSc Robotics and Computation

Supervisors: Dr.Baoru Huang, Dr.Francisco Vasconcelos

Submission date: September 2024

<sup>1</sup>**Disclaimer:** This report is submitted as part requirement for the MY DEGREE at UCL. It is substantially the result of my own work except where explicitly indicated in the text. The report will be distributed to the internal and external examiners, but thereafter may not be copied or distributed except with permission from the author.

## Abstract

3D reconstruction and image synthesis from endoscopic videos are essential to disease diagnosis inside human body, as well as downstream surgical applications, such as robotic-assisted minimally invasive surgery. Due to the nature of endoscopy that the monocular endoscope can only conduct one scan of the organ with a constrained endoscope view, however, the data collected are often constrained, which poses challenges to 3D reconstruction and image synthesis. Methods exist based on famous Neural Radiance Fields (NeRF) and 3D Gaussian Splatting. Most of them focus on refurbishing the models with surface alignment techniques, in order to smooth the reconstructed 3D surface. However, in this work, we explore the integration of diffusion models with 3D Gaussian Splatting for high-fidelity endoscopic image synthesis, specifically targeting colonoscopy applications, utilizing the diffusion model’s capability of generating images from noise. It is expected to address the challenges posed by limited data and constrained camera angles.

The diffusion model we choose is the Denoising Diffusion Implicit Model (DDIM), which is designed to generate data from novel views while maintaining consistency and accuracy, upholding the rigour required in medical applications. We also incorporate Signed Distance Function (SDF) regularization to better constrain the distribution of Gaussians, ensuring the synthesized models conform to the organ’s surface.

The experiments demonstrate that incorporating DDIM improves both Peak Signal-to-Noise Ratio (PSNR) and structural coherence without compromising the precision required for medical applications. The work can be extended through further optimization over hyper-parameters and refinements over codes to enhance the efficiency and reliability of this approach in clinical settings.

# Contents

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                        | <b>2</b>  |
| 1.1      | Endoscopy . . . . .                                        | 2         |
| 1.2      | Colonoscopy . . . . .                                      | 2         |
| 1.3      | Novel view images . . . . .                                | 2         |
| 1.4      | Project Overview . . . . .                                 | 3         |
| <b>2</b> | <b>Literature Review</b>                                   | <b>5</b>  |
| 2.1      | INR and NeRF . . . . .                                     | 5         |
| 2.2      | 3D Gaussian Splatting . . . . .                            | 5         |
| 2.3      | Diffusion models . . . . .                                 | 6         |
| 2.4      | NeRFStudio & Diffuser . . . . .                            | 6         |
| <b>3</b> | <b>Background</b>                                          | <b>7</b>  |
| 3.1      | 3D Gaussian Splatting Optimization and Rendering . . . . . | 7         |
| 3.1.1    | Architecture . . . . .                                     | 7         |
| 3.1.2    | Structure from Motion . . . . .                            | 7         |
| 3.1.3    | 3D Gaussian to 2D Projection . . . . .                     | 8         |
| 3.1.4    | Spherical Harmonics . . . . .                              | 9         |
| 3.1.5    | Optimization . . . . .                                     | 10        |
| 3.1.6    | Rasterization . . . . .                                    | 11        |
| 3.2      | Diffusion Models . . . . .                                 | 11        |
| 3.2.1    | Denoising Diffusion Probabilistic Models . . . . .         | 11        |
| 3.2.2    | Denoising Diffusion Implicit Models . . . . .              | 12        |
| 3.3      | Score Jacobian Chaining . . . . .                          | 14        |
| 3.3.1    | Score Based Diffusion . . . . .                            | 14        |
| 3.3.2    | Score Jacobian Chaining: 2D score to 3D score . . . . .    | 14        |
| 3.3.3    | Out-of-distribution Problem . . . . .                      | 14        |
| 3.4      | Signed Distance Function . . . . .                         | 14        |
| 3.5      | Evaluation Matrices . . . . .                              | 15        |
| <b>4</b> | <b>Methodology</b>                                         | <b>16</b> |
| 4.1      | Overview . . . . .                                         | 16        |
| 4.2      | SfM Initialization and 3DGS . . . . .                      | 17        |
| 4.3      | DDIM-Splatting . . . . .                                   | 17        |
| 4.3.1    | Loss Function . . . . .                                    | 17        |
| 4.3.2    | Novel-view Image Generation . . . . .                      | 18        |
| 4.3.3    | Diffusion Model . . . . .                                  | 18        |

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| 4.4      | SDF Regularization . . . . .                    | 18        |
| <b>5</b> | <b>Experiments</b>                              | <b>20</b> |
| 5.1      | Dataset . . . . .                               | 20        |
| 5.2      | DDIM and DDPM Model . . . . .                   | 20        |
| 5.3      | Gaussian Splatting and instant-NGP . . . . .    | 21        |
| 5.4      | DDIM-Splatting and SDF-DDIM-Splatting . . . . . | 21        |
| <b>6</b> | <b>Results and Discussion</b>                   | <b>22</b> |
| 6.1      | DDIM-Splatting Ablation . . . . .               | 22        |
| 6.2      | Multiple Methods Ablation . . . . .             | 24        |
| 6.3      | Novel-view Sampling Ablation . . . . .          | 25        |
| 6.4      | SDF Regularization Level Ablation . . . . .     | 26        |
| <b>7</b> | <b>Conclusion and Future Work</b>               | <b>29</b> |

# List of Figures

|      |                                                                                                            |    |
|------|------------------------------------------------------------------------------------------------------------|----|
| 1.1  | 3D Gaussian Splatting Rendered Image from Novel View . . . . .                                             | 3  |
| 1.2  | Diffused Clean Image from Novel View . . . . .                                                             | 3  |
| 3.1  | Architecture of 3D Gaussian Splatting (3DGS) [1]: . . . . .                                                | 7  |
| 3.2  | Structure from Motion pipeline in COLMAP[2] . . . . .                                                      | 8  |
| 3.3  | Transforming from camera space to image space [3] . . . . .                                                | 9  |
| 3.4  | Alternative picture for the real spherical harmonics $Y_{\ell m} = \Theta(\theta)\Phi(\phi)$ [4] . . . . . | 10 |
| 3.5  | Adaptive Control Of Gaussian [1] . . . . .                                                                 | 10 |
| 4.1  | DDIM-Splatting Main Pipeline. . . . .                                                                      | 16 |
| 4.2  | Comparison of DDPM and DDIM at different steps. . . . .                                                    | 18 |
| 6.1  | GPU Run-time, . . . . .                                                                                    | 22 |
| 6.2  | Chart 1: . . . . .                                                                                         | 23 |
| 6.3  | Chart 2: . . . . .                                                                                         | 23 |
| 6.4  | Ablation Study 1 . . . . .                                                                                 | 23 |
| 6.5  | Ablation Study 1 Table Comparison: . . . . .                                                               | 23 |
| 6.6  | Chart 1: . . . . .                                                                                         | 24 |
| 6.7  | Chart 2: . . . . .                                                                                         | 24 |
| 6.8  | Ablation Study 2: . . . . .                                                                                | 24 |
| 6.9  | Ablation Study 2 Table Comparison: . . . . .                                                               | 24 |
| 6.10 | Renders: . . . . .                                                                                         | 25 |
| 6.11 | Chart 1: . . . . .                                                                                         | 26 |
| 6.12 | Chart 2: . . . . .                                                                                         | 26 |
| 6.13 | Ablation Study 3: . . . . .                                                                                | 26 |
| 6.14 | Ablation Study 3 Table Comparison: . . . . .                                                               | 26 |
| 6.15 | Chart 1: . . . . .                                                                                         | 27 |
| 6.16 | Chart 2: . . . . .                                                                                         | 27 |
| 6.17 | Ablation Study 4: . . . . .                                                                                | 27 |
| 6.18 | Ablation Study 4 Table Comparison: . . . . .                                                               | 27 |
| 6.19 | Gaussian count: . . . . .                                                                                  | 28 |
| 6.20 | Frames per Second: . . . . .                                                                               | 28 |

# List of Tables

|                                        |    |
|----------------------------------------|----|
| 5.1 Ablation Study Settings, . . . . . | 21 |
|----------------------------------------|----|

# Abbreviations

**CRC** Colorectal Cancer

**C3VD** Colonoscopy 3D Video Dataset

**INR** Implicit Neural Representation

**NeRF** Representing Scenes as Neural Radiance Fields for View Synthesis

**MLPs** Multi-Layer Perceptrons

**3DGS** 3D Gaussian Splatting

**SfM** Structure from Motion

**DDPM** Denoising Diffusion Probabilistic Model

**DDIM** Denoising Diffusion Implicit Model

**SJC** Score Jacobian Chaining

**GAN** Generative Adversarial Network

**SIFT** scale-invariant feature transform

**PDF** Probability Density Function

**SSIM** Structural Similarity Index Measure

**PPAS** Perturb-and-Average Scoring

**SDF** Signed Distance Function

**PSNR** Peak Signal-to-Noise Ratio

**FPS** Frames Per Second

**LPIPS** Learned Perceptual Image Patch Similarity

# Chapter 1

## Introduction

### 1.1 Endoscopy

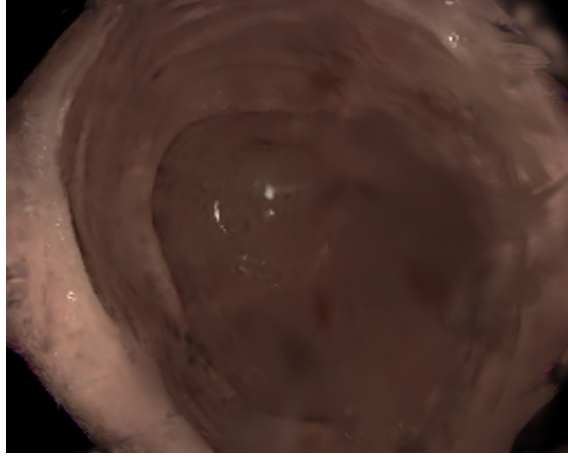
Endoscopy refers to the procedure where a thin and flexible tube equipped with a camera is inserted into the human body for examination of potential abnormalities, via the images captured by the camera. Many types are familiar to the public, including gastroscopy, colonoscopy, and oral endoscopy, among others [5]. It is important to note that, due to the use of the body’s natural openings for examinations, the devices employed must be compact, typically equipped with only monocular cameras. This limitation presents significant challenges, particularly in terms of depth prediction and, consequently, 3D reconstruction. Accurate 3D models are essential for assisting in precise disease diagnosis and surgical planning within the human body. During an endoscopy, the camera can only traverse the organ once to collect data, leading to potential misdiagnoses due to the limited number of images and restricted viewing angles. Furthermore, traditional geometry-based approaches are often unsuitable, as organ surfaces consist of soft tissue with minimal distinct features and repetitive textures [5]. Thus, more advanced solutions are required to address these challenges effectively.

### 1.2 Colonoscopy

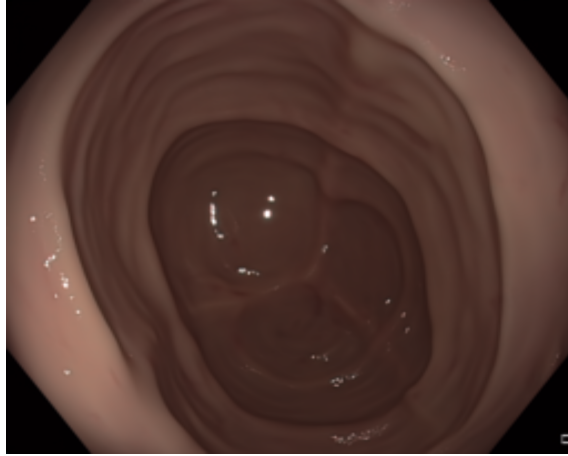
Colonoscopy is regarded as a typical form of endoscopy and a fundamental method for detecting polyps and diagnosing Colorectal Cancer (CRC) [6], which is a leading cause of mortality, ranking third after lung and breast cancer in women and lung and prostate cancer in men [7]. Thus, the project aims to improve existing approaches to novel view synthesis in endoscopy using Colonoscopy 3D Video Dataset (C3VD) [8], an open-source dataset. Although C3VD is not a real colonoscopy dataset, it remains the only publicly available endoscopy dataset that combines photorealistic colon images with accurate ground truth regarding endoscope motion and the 3D shape of a colon phantom. Therefore, it nonetheless serves as a valid and reasonable choice for evaluating Gaussian splatting and other methods in endoscopic 3D reconstruction and image synthesis.

### 1.3 Novel view images

While constructing 3D models from captured endoscopy images is possible with popular methods such as 3DGS, novel-view synthetic images provide additional perspectives beyond the original



*Figure 1.1: 3D Gaussian Splatting Rendered Image from Novel View*



*Figure 1.2: Diffused Clean Image from Novel View*

trajectory, serving as valuable priors for 3D model reconstruction[9]. Furthermore, novel view synthesis from 3D models can offer more comprehensive visual information, helping to reduce blind spots and enhance diagnostic accuracy in complex medical procedures. For instance, it can assist in identifying areas that may contain polyps or other abnormalities that might be missed with traditional single-view endoscopy, thus improving the overall effectiveness of the examination.

After reviewing colonoscopy videos and testing the 3DGS model, I observed that it successfully generates reasonable 3D models with smooth surfaces and intact structures, consistent with the content of each video. However, the model struggles to render views that deviate from the original camera trajectory, even with slight variations, as shown in Figure 1.1. Given the recent popularity of diffusion models for their ability to generate images from noise, I trained and tested a diffusion model on the same data in Figure 1.1. The results, displayed in Figure 1.2, show a stark contrast between the two images. By integrating diffusion models with 3D Gaussian Splatting, colonoscopy image synthesis shows promising potential for enhanced performance.

## 1.4 Project Overview

The project investigates the integration of diffusion models into 3DGS for novel view image synthesis. It begins by revisiting the fundamentals of 3D Gaussian Splatting and diffusion models.

Through this review, we identify suitable libraries and diffusion models to build the pipeline for the DDIM-Splatting model. The primary goal is to determine whether incorporating the diffusion model enhances image synthesis or merely introduces additional noise. The specific objectives of the project are as follows::

- Understand the principles behind 3D Gaussian Splatting and modern diffusion models;
- Select an appropriate diffusion model that enables fast and accurate inference;
- Develop a model-training pipeline that integrates the chosen diffusion model;
- Evaluate the impact of the diffusion model on image synthesis quality;
- Optimize the pipeline to improve performance following the integration of the new feature.

## Chapter 2

# Literature Review

The method I propose primarily leverages Gaussian splatting and diffusion models. Gaussian splatting, introduced after Representing Scenes as Neural Radiance Fields for View Synthesis (NeRF)—an implicit neural representation method known for state-of-the-art results—offers comparable rendering quality but with significantly faster performance. Diffusion models, which have gained popularity in recent years, have been successfully combined with NeRF and 3DGS to enhance image generation tasks. Given these advancements, this work reviews studies related to NeRF, 3DGS, diffusion models, as well as the two key libraries employed for implementation.

### 2.1 INR and NeRF

Conventionally, discretized data are used to represent our visual world, for instance, point clouds, voxel grids, and meshes, which follow an explicit paradigm [10]. In contrast, Implicit Neural Representation (INR) has been explored in recent years, allowing to mapping coordinates in continuous space to corresponding values, like RGB or opacity values. Multi-Layer Perceptrons (MLPs) are often trained as mapping functions, and NeRF just combines both fully connected MLPs, i.e. a modern feedforward artificial neural network, and volume rendering technique to implicitly model scenes and objects [10], which allows to represent continuous scenes with complex geometry and materials in an storage-efficient way [11].

Moreover, it has been studied intensively these years for assisting endoscopy, and EndoNerf [12] firstly exploited Nerf in surgical scene reconstruction. Besides the incorporation of Nerf, it also designed a mask-guided ray-casting strategy to solve tool occlusion in surgery and use depth data for optimization. EndoSurf [13] applies three MLPs to model deformation, geometry and appearance separately, giving even better outcomes. Additionally, Huang et al. [14] utilizes a pre-trained depth estimator trained on nature scenes further improving the results.

### 2.2 3D Gaussian Splatting

In addition to basic implicit NeRF-based methods, 3D Gaussian Splatting with explicit representation, i.e. 3DGS [1] is also a popular method studied for endoscopy image generation and 3D model reconstruction. With 3DGS, a collection of 3D Gaussians are used for 3D environment representation. While those Gaussians allow for differentiable volumetric representations in continuous space, they are unstructured and explicit for fast rendering simultaneously. Plenty of methods are proposed for improvement over the 3DGS model, considering its promising capability and some

of the limitations still present in 3D Gaussian Splatting techniques. For instance, SuGaR[15] and GSDF [16] introduce Signed Distance Function (SDF) as a geometry regularization to enhance image quality.

In the realm of endoscopic video analysis, EndoGS [17], as the first method based on 3D Gaussian Splatting for deformable endoscopic tissue reconstruction, solves the tool occlusion problem via incorporating time dimensional data and capturing better geometry via combining surface-aligned regularization terms. However, it tends to EXHIBIT artifacts in untrained views and has strict requirements for starting points, as noted by Bonilla et al. in their work on Gaussian Pancakes [18]. To overcome these challenges, they introduced Gaussian Pancakes, which synergizes geometrically-regularized 3D Gaussian Splatting with RNNSLAM [19]. Yang et al. [20] further integrate 3D Gaussian Splatting and the MPM framework for efficient and accurate surgical scene simulation, addressing overfitting issues and enabling high-fidelity, near real-time soft tissue interactions.

Despite the development of these novel methods, image reconstruction continues to face challenges in terms of accuracy, realism, and training time, especially when constrained by the lack of comprehensive datasets. Thus, there remains a need to develop more efficient and reliable methods to tackle these persistent issues.

Innovative approaches are proposed by GaussianDiffusion [21], and ReconFusion [9], which integrated 3DGS and NeRF with diffusion models. On the one hand, GaussianDiffusion enables to generate 3D models that is consistent in multi-view geometry from text prompts, with the fusion of 3DGS and Denoising Diffusion Probabilistic Model (DDPM). By integrating Structured Noise and leveraging Variational Gaussian Splatting, this method addresses the inconsistencies between multi-view 2D diffusion models and 3D geometric modeling. On the other hand, ReconFusion utilize a Latent Diffusion Model to generate novel view images, using both real and artificial images for NeRF training. Thereby, the synthesized images serve as priors for novel view synthesis. Inspired by GaussianDiffusion and ReconFusion, Inspired by these methods, I propose a novel pipeline integrating a diffusion model with 3DGS, focusing on colonoscopy image synthesis experiments.

## 2.3 Diffusion models

Diffusion models are a type of probabilistic generative model which is capable of generating high-quality samples. One of the most well-known models is Generative Adversarial Network (GAN) [22] for generating images, trained with adversarial training. Recently, plenty of studies have been conducted on diffusion models, such as DDPM [23], Denoising Diffusion Implicit Model (DDIM) [24], and Stable Diffusion [25]. Moreover, they are combined with NeRF methods to improve novel view synthesis and 3D model reconstruction: NerfDiff [26], Score Jacobian Chaining (SJC) [27], Reconfusion [9], and GaussianDiffusion [21]. In the following works, we will mainly explore methods described in GaussianDiffusion.

## 2.4 NeRFStudio & Diffuser

NeRFStudio [28] and Diffuser [29] are great libraries and frameworks for easy use of NeRF-based models and diffusion pipelines respectively. Not only do they provide pre-built commands and pipelines, but also allow for user customization, enabling users to design their functions while still benefiting from the convenience of training within these frameworks. As a result, the models and pipelines developed in this project are built upon these robust frameworks.

# Chapter 3

## Background

To lay the foundation for understanding the method, we briefly review the algorithms and principles behind it. It comprises all principles we need to know for the project, from existing models, including 3DGS and diffusion models, to individual methods: SJC and SDF.

### 3.1 3D Gaussian Splatting Optimization and Rendering

#### 3.1.1 Architecture

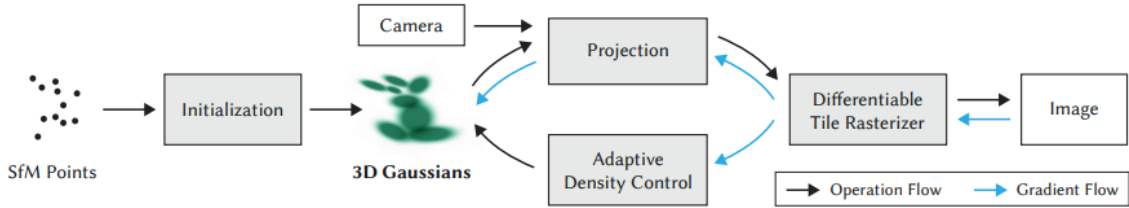


Figure 3.1: Architecture of 3DGS [1]:

Initializing 3D Gaussians from sparse SfM point cloud, 3DGS renders images with the novel tile-based renderer and train with adaptive density control over Gaussians.

As it is shown in Figure 3.1, 3DGS utilizes Structure from Motion (SfM) to initialize the position of 3D Gaussians, which is then ready to be rendered into images from different views via differentiable tile rasterizer, and optimized with adaptive density control. In the following subsections, details included in 3DGS will be introduced.

#### 3.1.2 Structure from Motion

3DGS requires SfM to provide sparse point clouds to initialize the Gaussians and then refine them. Here we will review how it works.

The procedure of simultaneously acquiring monocular camera poses and reconstructing a 3D scene is called SfM. It consists of feature matching, outlier rejection, camera pose estimation and bundle adjustment. For this project, COLMAP is used, which offers an automatic and easy-to-use reconstruction tool, taking images and generating point-cloud reconstruction. The pipeline used in COLMAP for SfM is demonstrated in Figure 3.2. It uses scale-invariant feature transform (SIFT) [30] for feature matching. The correspondences are then used for calculating the fundamental

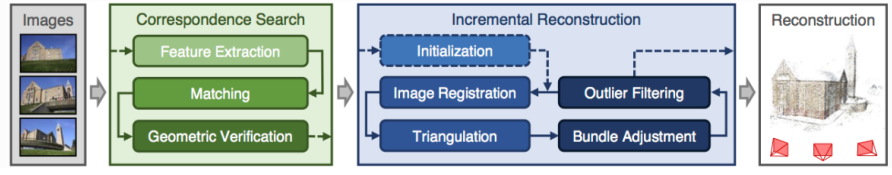


Figure 3.2: Structure from Motion pipeline in COLMAP[2]

matrix for removing outliers and obtaining camera poses. Finally, it applies bundle adjustment on all the camera poses obtained previously.

#### Bundle Adjustment

Bundle adjustment works by minimising a cost function that is related to a weighted sum of squared reprojection errors.

### 3.1.3 3D Gaussian to 2D Projection

Gaussians in 3DGS are defined as it is provided in EWA volume splatting [3]. The Probability Density Function (PDF) for 1-D Gaussian is defined as:

$$G(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

, where  $\mu$  is the mean of the distribution;  $\sigma^2$  is the variance. The PDF for the k-dimensional Gaussian distribution is given by:

$$G(\mathbf{x}) = \frac{1}{\sqrt{(2\pi)^k |\Sigma|}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$$

, where  $\mathbf{x}$  is a k-dimensional vector;  $|\Sigma|$  is the determinant of the covariance matrix  $\Sigma$ ;  $\Sigma^{-1}$  is the inverse of  $\Sigma$ ;  $(\mathbf{x} - \boldsymbol{\mu})^\top$  is the transpose of the vector difference. Gaussians are defined by  $\Sigma = RSS^\top R^\top$  in world space, so they need to transform from world coordinate system to camera coordinate system and then to image coordinate system by converting  $\Sigma$  to

$$\Sigma' = JW\Sigma W^T J^T$$

, where  $J$  is the Jacobian (first derivative of a matrix) of projective transformation from camera to image coordinate system;  $W$  is a viewing transformation matrix from world to camera coordinate system. The reasoning is given below.

Figure 3.3 depicts the fact that it is non-linear to transform from camera to image space. Additionally, the projective transformation function  $\phi(\mathbf{u})$  mapping point  $\mathbf{u}_k = [\hat{\mathbf{u}}, u_2]^\top$  to  $\mathbf{x}_k = [\hat{\mathbf{x}}, x_2]^\top$ , where  $\hat{\mathbf{u}} = [u_0, u_1]^\top$ , and  $\hat{\mathbf{x}} = [x_0, x_1]^\top$ , is

$$\begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \phi(\mathbf{u}) = \begin{pmatrix} u_0/u_2 \\ u_1/u_2 \\ \left\| \begin{pmatrix} u_0 \\ u_1 \\ u_2 \end{pmatrix}^T \right\| \end{pmatrix}.$$

With Taylor series expansion theory:

$$f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \frac{f'''(a)}{3!}(x-a)^3 + \dots + \frac{f^{(n)}(a)}{n!}(x-a)^n + \dots$$

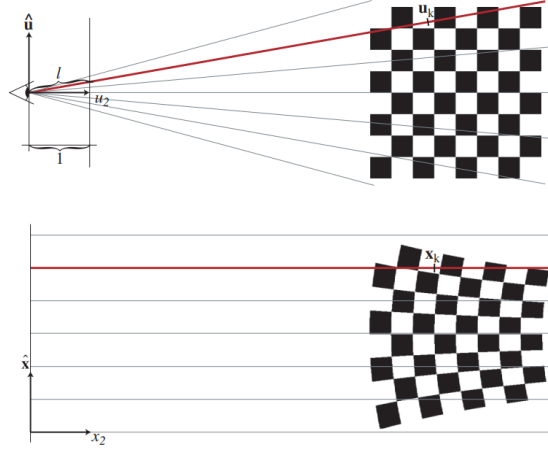


Figure 3.3: Transforming from camera space to image space [3]

, we can convert  $\phi(\mathbf{u})$  only using the first-order Taylor expansion equation.

$$\phi_k(\mathbf{u}) = \phi(\mathbf{u}_k) + \mathbf{J}_k \cdot (\mathbf{u} - \mathbf{u}_k)$$

$$\mathbf{J}_k = \frac{\partial \phi}{\partial \mathbf{u}}(\mathbf{u}_k) = \begin{pmatrix} 1/u_{k,2} & 0 & -u_{k,0}/u_{k,2}^2 \\ 0 & 1/u_{k,2} & -u_{k,1}/u_{k,2}^2 \\ u_{k,0}/l' & u_{k,1}/l' & u_{k,2}/l' \end{pmatrix}$$

, where  $l' = \left\| \begin{pmatrix} u_{k,0}, u_{k,1}, u_{k,2} \end{pmatrix}^T \right\|$ . Besides, viewing transformation function is defined as

$$\varphi(\mathbf{v}) = \mathbf{W}\mathbf{v} + \mathbf{d}$$

, and to conclude,

$$\mathbf{x} = \mathbf{m}_k(\mathbf{v}) = \phi_k(\varphi(\mathbf{v})) = \mathbf{J}_k \mathbf{W}\mathbf{v} + \mathbf{x}_k + \mathbf{J}_k(\mathbf{d} - \mathbf{u}_k).$$

According to the property of covariance, we can derive the transformed Gaussian is defined with  $\Sigma' = \mathbf{J}\mathbf{W}\Sigma\mathbf{W}^\top \mathbf{J}^\top$ .

### 3.1.4 Spherical Harmonics

Spherical harmonics are functions defined on the surface of a sphere [4], taking an angle as input, ignoring the distance in the space, and outputting the value on the sphere surface. The solutions for  $Y_\ell^m(\theta, \phi)$  are the spherical harmonics:

$$P_\ell^m(x) = \frac{(-1)^m}{2^\ell \ell!} (1-x^2)^{m/2} \frac{d^{\ell+m}}{dx^{\ell+m}} (x^2-1)^\ell.$$

$$Y_\ell^m(\theta, \phi) = \sqrt{\frac{2\ell+1}{4\pi} \frac{(\ell-m)!}{(\ell+m)!}} P_\ell^m(\cos \theta) e^{im\phi}.$$

, where  $(\theta, \phi)$  is an angle input;  $\theta$  indicates polar angle;  $\phi$  indicates azimuthal angle;  $Y_\ell^m(\theta, \phi)$  with different  $\ell$  are categorized into different bands, as it is shown in Figure 3.4.

| l: |   | $P_\ell^m(\cos \theta) \cos(m\varphi)$                                            | $P_\ell^{ m }(\cos \theta) \sin( m \varphi)$                                      |                                                                                    |
|----|---|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| 0  | s |  |                                                                                   |  |
| 1  | p |  |  |                                                                                    |
| 2  | d |  |  |                                                                                    |
| 3  | f |  |  |                                                                                    |
| 4  | g |  |  |                                                                                    |
| 5  | h |  |  |                                                                                    |
| 6  | i |  |  |                                                                                    |
| m: |   | 6 5 4 3 2 1 0                                                                     | -1 -2 -3 -4 -5 -6                                                                 |                                                                                    |

Figure 3.4: Alternative picture for the real spherical harmonics  $Y_{\ell m} = \Theta(\theta)\Phi(\phi)$  [4]

It can be used to compute the colours of objects viewed from different angles, using

$$c = \sum_{l=0}^{l_{\max}} \sum_{m=-l}^l k_l^m Y_l^m(\theta, \phi)$$

, expressing the colour  $C$  as a weighted sum of Spherical Harmonics functions, where  $k_l$  is the weight for each  $Y$ .

### 3.1.5 Optimization

The loss is defined by

$$\mathcal{L} = (1 - \lambda)\mathcal{L}_1 + \lambda\mathcal{L}_{\text{D-SSIM}}$$

, consisting of a weighted L1 loss (Mean Absolute Error) and Structural Similarity Index Measure (SSIM). Except for back-propagation, it also proposes adaptive control of Gaussians, algorithms shown in Figure 3.5.

```

if IsRefinementIteration(i) then
  for all Gaussians  $(\mu, \Sigma, c, \alpha)$  in  $(M, S, C, A)$  do
    if  $\alpha < \epsilon$  or IsTooLarge( $\mu, \Sigma$ ) then ▷ Pruning
      RemoveGaussian()
    end if
    if  $\nabla_p L > \tau_p$  then ▷ Densification
      if  $\|S\| > \tau_S$  then ▷ Over-reconstruction
        SplitGaussian( $\mu, \Sigma, c, \alpha$ )
      else ▷ Under-reconstruction
        CloneGaussian( $\mu, \Sigma, c, \alpha$ )
      end if
    end if
  end for
end if

```

Figure 3.5: Adaptive Control Of Gaussian [1]

, defines how Gaussian Splatting refine the model after back propagating the loss, by either pruning or densifying.

It prunes Gaussians with alpha values less than a certain threshold; duplicates for under-reconstruction cases, where Gaussian sizes are too small; and splits for over-reconstruction cases, where Gaussian sizes are too large.

### 3.1.6 Rasterization

The rasterization algorithm given in 3DGS implement a novel tile-based rasterizing method, splitting a screen into 16 x 16 tiles, culling and sorting Gaussians, and  $\alpha$ -blending for each pixel. The blending technique is the same as NeRF’s-point-based  $\alpha$ -blending. The colour is given by:

$$C = \sum_{i \in \mathcal{N}} c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j),$$

where  $c_i$  and  $\alpha_i$  are the colour and transparency of the pixel respectively.  $\alpha$  is computed from 2D Gaussian with covariance  $\Sigma$  and a learned opacity[31].

## 3.2 Diffusion Models

### 3.2.1 Denoising Diffusion Probabilistic Models

DDPM made some improvements on diffusion probabilistic models which add noise to images and then predict the original images. Differing from traditional generative models, in DDPM, the generation process is defined as the reverse of the Markov diffusion process, and at each step of the reverse sampling process, the model predicts the noise, reducing the learning difficulties.

In its forward process, we transform a clean and meaningful image to Gaussian noise by adding noise to the previous image at each step, but this process is inefficient. Therefore, a computational trick of the conditional Gaussian distribution can be used based on Markovian properties to obtain  $x_t$  noisy image at any time-step directly from the one-step calculation. This process can be represented as conditional distribution :

$$q(x_t | x_0) \sim \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I) \quad .$$

Therefore, the formula for  $x_t$  sampling is :

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon_t, \quad \bar{\alpha}_t = \prod_{i=1}^t \alpha_i, \quad \epsilon_t \sim \mathcal{N}(0, I) \quad .$$

Knowing  $x_t$  represented by  $x_0$  and  $\epsilon_t$ , we can then represent  $x_0$ , the clean image we want to deduce from the noise, with  $x_t$  and  $\epsilon_t$ :

$$x_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t}\epsilon_t}{\sqrt{\bar{\alpha}_t}}, \quad \epsilon_t \sim \mathcal{N}(0, I).$$

This inverse process is called the denoising process, which is a chain decomposition of the joint probability  $p(x_{0:T})$  according to the inverse process:

$$p(x_{0:T}) = p(x_T) \prod_{t=T}^1 p(x_{t-1} | x_t).$$

$p(x_T) \sim \mathcal{N}(0, I)$  is a standard normal distribution with mean 0 and identity covariance  $I$ , while  $p(x_{t-1}|x_t)$  is the conditional distribution of the reverse process, which shows how  $x_t$  is transformed into  $x_{t-1}$  by removing part of the Gaussian noise to obtain  $x_{t-1}$ . The primary goal is to approximate  $p(x_{t-1}|x_t)$ . Based on the maximum likelihood estimation theory, we need to

maximize the likelihood of the observed data  $\ln p(x_0)$ . However, the latent variables  $x_{1:T}$  are sampled over the entire network, and this leads to difficulty in computing the marginal probability of  $p(x_0)$ . Instead, we need to approximate this marginal probability by transforming the latent variables into the conditional probability of  $p(x_0)$  and then marginalizing the entire sequence:

$$\ln p(x_0) = \ln \int p(x_{0:T}) dx_{1:T}$$

Since we do not have the observed data  $x_{1:T}$ , which is considered latent, this is a typical hidden variable model estimation problem. As shown in the equation, because the latent variables exist, the likelihood can only be approximated. This requires transforming  $x_{1:T}$  into an easier form to estimate. However, because direct computation is difficult, we use Jensen's inequality to derive the Evidence Lower Bound (ELBO). The ELBO provides a lower bound for the log-likelihood, and under certain conditions, maximizing the ELBO is equivalent to maximizing the original likelihood.

$$\begin{aligned} \ln p(x_0) &= \ln \int p(x_{0:T}) dx_{1:T} \\ &= \ln \int \frac{p(x_{0:T})q(x_{1:T}|x_0)}{q(x_{1:T}|x_0)} dx_{1:T} \\ &= \mathbb{E}_{q(x_{1:T}|x_0)} \left[ \ln \frac{p(x_{0:T})}{q(x_{1:T}|x_0)} \right] \\ &\geq \mathbb{E}_{q(x_{1:T}|x_0)} [\ln p(x_0)] - \sum_{t=2}^T \mathbb{E}_{q(x_t|x_0)} [D_{KL}(q(x_{t-1}|x_t, x_0) || p_\theta(x_{t-1}|x_t))] \end{aligned}$$

After substitution, the final objective function becomes a simple least squares loss. Let us denote this loss by  $L_\gamma$ , where  $\gamma_t$  represents some weighting factor, influencing the final optimization result.

$$L_\gamma = \sum_{t=1}^T \gamma_t \mathbb{E}_{\epsilon_t} \left[ \|\epsilon_t - \hat{\epsilon}_\theta(x_t, t)\|^2 \right], \quad \epsilon_t \sim \mathcal{N}(0, I)$$

### 3.2.2 Denoising Diffusion Implicit Models

By reviewing the Denoising Diffusion Probabilistic Models (DDPM), we understand that the generative process follows a Markovian process, effectively reversing the Markovian training process. However, this characteristic results in the need for a large number of iterations during inference, leading to longer inference times. DDIM propose to utilize non-Markovian inference to largely reduce the time required by 10 to 50 times [24]. DDIM explores the non-Markovian property by redefining  $q(x_{1:T} | x_0)$  as

$$q_\sigma(x_{1:T} | x_0) := q_\sigma(x_T | x_0) \prod_{t=2}^T q_\sigma(x_{t-1} | x_t, x_0)$$

, with  $q_\sigma(x_T | x_0)$  being the same in DDPM:

$$q_\sigma(x_T | x_0) \sim \mathcal{N}(\sqrt{\bar{\alpha}_T} x_0, (1 - \bar{\alpha}_T)I)$$

, while in original DDPM, it is defined as

$$q(x_{1:T} | x_0) := q(x_1 | x_0) \prod_{t=2}^T q(x_t | x_{t-1}).$$

In the process of deriving DDPM loss, i.e. the differences between  $\hat{\epsilon}_\theta(x_t, t)$  and  $\epsilon$ , we also derive

$$q(x_{t-1} | x_t, x_0) \propto \mathcal{N} \left( x_{t-1}; \underbrace{\frac{\sqrt{\bar{\alpha}_t}(1 - \bar{\alpha}_{t-1})x_t + \sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)x_0}{1 - \bar{\alpha}_t}}_{\mu_q(x_t, x_0)}, \underbrace{\frac{(1 - \alpha_t)(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} I}_{\Sigma_q(t)} \right).$$

In order to leverage the non-Markovian property but still use the same model as DDPM, the expression of  $q(x_t | x_0)$  and  $q(x_{t-1} | x_t, x_0)$  should be the same as DDPM. Then, we redefine

$$q_\sigma(x_{t-1} | x_t, x_0) \sim \mathcal{N} \left( \frac{\sqrt{\bar{\alpha}_{t-1}}x_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \cdot \frac{x_t - \sqrt{\bar{\alpha}_t}x_0}{\sqrt{1 - \bar{\alpha}_t}}, \sigma_t^2 I \right)$$

for any  $t > 1$ , and  $q_\sigma(x_T | x_0)$  being the same as DDPM:

$$q_\sigma(x_T | x_0) \sim \mathcal{N}(\sqrt{\bar{\alpha}_T}x_0, (1 - \bar{\alpha}_T)I) \quad .$$

With

$$\hat{x}_0 = f_\theta^{(t)}(x_t) = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \hat{\epsilon}_t(x_t, t)}{\sqrt{\bar{\alpha}_t}}$$

, we can replace  $x_0$  again with  $x_t$  and  $\epsilon_t$ :

$$p_{\theta, \sigma}(x_{t-1} | x_t) \sim \mathcal{N} \left( \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \cdot \frac{x_t - \sqrt{\bar{\alpha}_t} \hat{x}_0}{\sqrt{1 - \bar{\alpha}_t}}, \sigma_t^2 I \right) \approx q_\sigma(x_{t-1} | x_t, x_0).$$

To conclude, the process of regeneration starts from sampling  $x_T$  from  $\mathcal{N}(0, I)$ , and denoising the image to generate a cleaner one with probability

$$p(x_{t-1} | x_t) = \begin{cases} \mathcal{N}(\hat{x}_0(x_1, t=1), \sigma_1^2 I) & \text{if } t = 1 \\ q_\sigma(x_{t-1} | x_t, \hat{x}_0(x_t, t)) & \text{if } 1 < t \leq T \end{cases}.$$

The formula for sampling  $x_{t-1}$  is given by:

$$\begin{aligned} x_{t-1} &= \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \cdot \frac{x_t - \sqrt{\bar{\alpha}_t} \hat{x}_0}{\sqrt{1 - \bar{\alpha}_t}} + \sigma_t \epsilon_t^* \\ &= \sqrt{\bar{\alpha}_{t-1}} \left( \underbrace{\frac{x_t - \sqrt{1 - \bar{\alpha}_t} \hat{\epsilon}_t(x_t, t)}{\sqrt{\bar{\alpha}_t}}}_{\text{predict } x_0} \right) + \underbrace{\sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \hat{\epsilon}_t(x_t, t)}_{\text{direction pointing to } x_t} + \underbrace{\sigma_t \epsilon_t^*}_{\text{random noise}} \end{aligned}$$

, where  $\epsilon_t^* \sim \mathcal{N}(0, I)$ .

In DDIM, the forward process is no longer Markovian, and  $q_\sigma(x_{t-1} | x_t, x_0)$  is no longer only dependent on the last state. Therefore the authors [24] call it a non-Markovian diffusion process. It is also possible to directly utilise a previously trained model of DDPM (with the same loss function) to predict noise without having to re-train a new model.

### 3.3 Score Jacobian Chaining

#### 3.3.1 Score Based Diffusion

Viewing the diffusion model from another perspective, it revolves around learning a denoiser  $D$  that minimizes the differences between a sample  $x$  from a dataset  $X = x_i$ , and the denoised sample  $D(x + \sigma n)$ , where  $n$  is sampled from a standard normal distribution:

$$\mathbb{E}_{x \sim p_{\text{data}}} \mathbb{E}_{n \sim \mathcal{N}(0, \mathbf{I})} \|D(x + \sigma n; \sigma) - x\|_2^2.$$

$D$  is chosen to be DDIM in this project:

$$D(x + \sigma n; \sigma).$$

With the learning target mentioned above, it can be used to approximate well the score function of the noised dataset distribution  $p_\sigma(x)$  at different noise levels via different  $\sigma$  [27]:

$$\nabla_x \log p_\sigma(x) \approx \frac{D(x; \sigma) - x}{\sigma^2}.$$

#### 3.3.2 Score Jacobian Chaining: 2D score to 3D score

After knowing the interpretation of the diffusion model from a score-function perspective, we can then talk about how to use the chain rule to extend the 2D diffusion model to 3D. In the setting of this project, we only have a pre-trained 2D diffusion model and want to calculate the score of the diffusion model concerning 3D asset  $\theta$ . Therefore, we can use the chain rule to first calculate the 2D score and then multiply it by the 3D rendering gradient. Recalling the score function is the gradient of the log probability density of data [27], we have the equation:

$$\underbrace{\nabla_\theta \log p_\sigma(\theta)}_{\text{3D score}} = \mathbb{E}_\pi \left[ \underbrace{\nabla_{x_\pi} \log p_\sigma(x_\pi)}_{\text{2D score; pretrained}} \cdot \underbrace{J_\pi}_{\text{renderer Jacobian}} \right].$$

#### 3.3.3 Out-of-distribution Problem

It is common to cause out-of-distribution problems when it comes to applying diffusion on non-noisy images. The reason is that the denoiser is trained on noisy inputs, so generating from a clean image does not consist of the input distribution. To tackle this problem, Wang in [27] proposes Perturb-and-Average Scoring (PPAS) that perturb the clean image before passing to the denoiser.

### 3.4 Signed Distance Function

Applying the diffusion model, although more information about the scene is obtained, the data is noisier, and so are the results, with Gaussians off the surface. SDF is a function taking a position as an input and outputs the distance from that position to the nearest part of a shape. For example, the most straightforward SDF is that of a 2D circle: the Euclidean distance to the centre of the circle minus the radius. In order to regularize the Gaussians to the surface of a scene, the difference between SDF of Gaussians  $f(p)$  and “ideal” SDF  $\hat{f}(p)$  is chosen to be minimized, where  $p$  is a position close to the surface of the scene. In this project, we implement the SDF-regularization approach mentioned in SuGaR [15], leveraging the Gaussian properties.

The SDF of a position  $p$  to Gaussians is calculated from a density function  $d(p) : \mathbf{R}^3 \rightarrow R_+$ :

$$d(p) = \sum_g \alpha_g \exp \left( -\frac{1}{2} (p - \mu_g)^\top \Sigma_g^{-1} (p - \mu_g) \right)$$

, where:

- $\alpha_g$  is the alpha-blending coefficients at position  $p$ ,
- $\mu_g$  is the mean vector of the  $g$ -th Gaussian,
- $\Sigma_g$  is the covariance matrix of the  $g$ -th Gaussian,
- $(p - \mu_g)^\top \Sigma_g^{-1} (p - \mu_g)$  is the Mahalanobis distance between  $p$  and the mean  $\mu_g$ , scaled by  $\Sigma_g$ .

In the practice, we replace Gaussians  $g$  with the “closest Gaussian”  $g^*$ , which has the largest contribution at position  $p$ :

$$g^* = \arg \min_g \{ (p - \mu_g)^\top \Sigma_g^{-1} (p - \mu_g) \}.$$

Then the SDF  $f(p)$  is

$$f(p) = \pm s_{g^*} \sqrt{-2 \log (d(p))}.$$

The density for ideal SDF  $\bar{d}(p)$  is:

$$\bar{d}(p) = \exp \left( -\frac{1}{2s_{g^*}^2} \langle p - \mu_{g^*}, n_{g^*} \rangle^2 \right)$$

, where:

- $s_{g^*}$  is the smallest scaling factor of the Gaussian
- $n_{g^*}$  is the direction of the corresponding axis.

Therefore, ideal SDF is

$$\bar{f}(p) = \pm s_{g^*} \sqrt{-2 \log (\bar{d}(p))}.$$

The regularization term  $\mathcal{R}$  is

$$\mathcal{R} = \frac{1}{|P|} \sum_{p \in P} |\hat{f}(p) - f(p)|$$

, where  $P$  is the set of sampled points.

### 3.5 Evaluation Matrices

Peak Signal-to-Noise Ratio (PSNR) measures the noise introduced during reconstruction by comparing reconstructed images with the original ones. The higher the PSNR values, the better the image quality and less noise. Frames Per Second (FPS) indicates the number of frames the system can process per second on average, which is a critical metric in real-time systems or speed performance evaluation of a model. SSIM considering changes in structural information, luminance and contrast, evaluates the similarity between images. It correlates better with how humans perceive image quality compared to L1 loss. Lastly, Learned Perceptual Image Patch Similarity (LPIPS) uses a deep learning model to measure the similarity between images in a feature space, which is even better than SSIM scores in mimicking human perception of visual quality. It is worth noticing that the images are of higher quality with lower LPIPS scores.

# Chapter 4

## Methodology

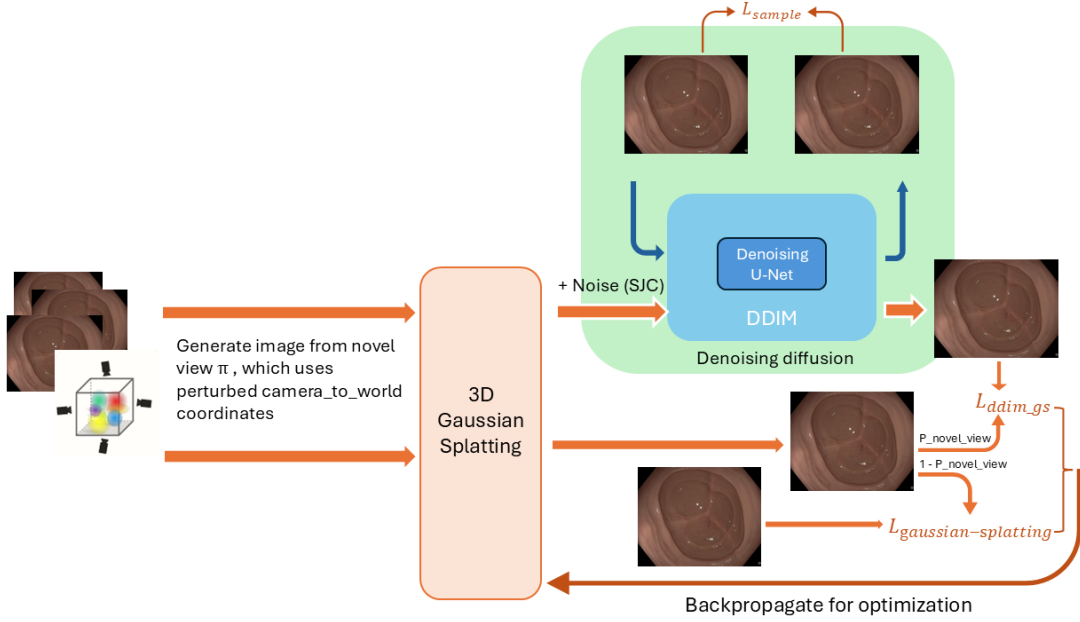


Figure 4.1: DDIM-Splatting Main Pipeline.

After pretraining the 3D Gaussian Splatting model for certain steps, we can train with this DDIM-Splatting pipeline. We randomly sample novel views for training (with probability  $P_{\text{novel\_view}}$ ). It starts with rendering with 3DGS model, and the rendered images are then processed by trained DDIM model to generate clean pseudo-ground truth. The loss for novel view training is  $L_{\text{ddim\_gs}}$ , while for original dataset training is  $L_{\text{gaussian-splatting}}$ .

### 4.1 Overview

Figure 4.1 illustrates the main pipeline of the DDIM-Splatting Model. Starting from 3D Gaussian Splatting model training, the original images are processed by Colmap[2, 32] to get camera coordinates and sparse 3D points to initialize 3D Gaussians in 3DGS model. After pre-training the 3DGS model for certain steps, we can train the DDIM-Splatting model. With specific probability we either choose original images for further training or generate novel view images and take the diffused images just generated as ground truth data for loss calculation and optimization.

In terms of novel-view image generation, we perturb the camera coordinates before any ren-

dering. Then, we render the image from the novel view with 3DGS, which is then utilized for ground-truth generation, as it is described in Score Jacobian Chaining [27]: perturbed with Gaussian noise and input into pre-trained DDIM model. Having acquired both the 3DGS rendered image and the artificial ground truth, the loss can be calculated and propagated through the model.

## 4.2 SfM Initialization and 3DGS

Known that 3DGS should be trained from SfM sparse points rather than raw images(Figure 3.1), we process data using the “process\_data.py” in nerfstudio, which performs feature extraction, feature matching and bundle adjustment using Colmap[2, 32], and then save the camera transforms attained from Colmap to a JSON file. Then we use the processed data to initialize the Gaussian points in 3DGS and perform training with nerfstudio commands.

## 4.3 DDIM-Splatting

In the project, we modified the model code and pipeline code for 3DGS to suit the DDIM-Splatting model and pipeline.

### 4.3.1 Loss Function

Modifying the 3DGS model for the DDIM-Splatting, when using images from novel views, we define the loss function as the following. Firstly, we calculate the weight for each loss:

$$\begin{aligned} total\_w &= 1 + \lambda_{ssim}, \\ weight &= \left[ \frac{1}{total\_w}, \frac{\lambda_{ssim}}{total\_w} \right]. \end{aligned}$$

Then the main loss for novel view images is

$$L_{main} = \lambda_{diffusion} * L_{diff},$$

where

$$L_{diff} = weight[0] * L1 + weight[1] * L_{SSIM},$$

$L_{diff}$  is the weighted sum of L1 loss and SSIM loss, and the regularization comprises scale regularization and optionally SDF loss:

$$L_{reg} = L_{scale\_reg}(+L_{sdf\_reg}).$$

Finally,  $L_{total}$ , the total loss is

$$L_{total} = L_{main} + L_{reg}.$$

The loss of using images from datasets is the same as 3DGS. In this case, the main loss is a weighted combination of L1 loss ( $L_{L1}$ ), similarity loss ( $L_{SSIM}$ ), and PSNR loss ( $L_{PSNR}$ ). The total weight is given by:

$$L_{total} = L_{main} + L_{reg}$$

, where

$$L_{main} = weight[0] * L1 + weight[1] * L_{SSIM} + weight[2] * L_{PSNR}$$

, and the weights are calculated by

$$weight = \left[ \frac{1}{total\_w}, \frac{\lambda_{ssim}}{total\_w}, \frac{\lambda_{psnr}}{total\_w} \right],$$

and

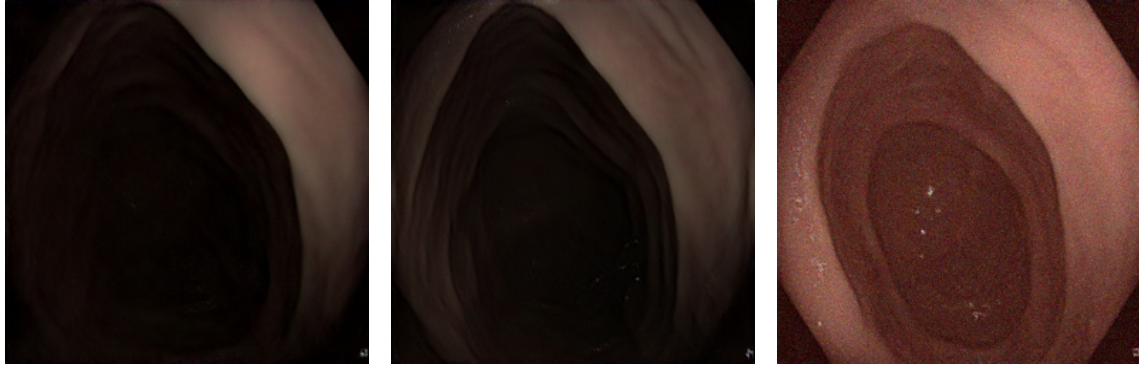
$$total\_w = 1 + \lambda_{ssim} + \lambda_{psnr}$$

### 4.3.2 Novel-view Image Generation

Modifying the pipeline for DDIM-Splatting, we generate new camera coordinates by perturbing original camera transforms from SfM with Gaussian noise, randomly with a certain probability. If the camera coordinate is new we replace the ground-truth image from datasets with the image that is first rendered from 3DGS and then perturbed and passed to the pre-trained DDIM model. Otherwise, we follow the traditional 3DGS pipeline.

### 4.3.3 Diffusion Model

In addition, the diffusion model is chosen to be a denoising diffusion implicit model rather than a denoising diffusion probability model due to the non-Markovian property the DDIM possesses. It guarantees the next denoised image is not only dependent on its last one but also on the original noisy image input into the model, which increases the reliability of the “ground-truth” image we generate. Illustrating in Figure 4.2, the DDIM model indeed inference a more reasonable image in fewer steps, much more efficient than DDPM model.



(a) 12 Steps DDPM

(b) 100 Steps DDPM

(c) 12 Steps DDIM

Figure 4.2: Comparison of DDPM and DDIM at different steps.

DDIM is able to produce a more reasonable image with fewer steps.

## 4.4 SDF Regularization

Having reviewed principles about how to derive SDF regularization values in Section 3.4 Chapter 3, it is time to discuss how to implement them in practice. Reminding that it measures the differences between SDF values that calculated from Gaussians and “ideal” SDF values. On the one hand, considering the first SDF value, the density on each point is required. We calculate it as a sum of local Gaussian opacities, and the local Gaussians are obtained using the “knn\_points” function

from PyTorch3D, which implements the K-nearest neighbours algorithm. The function returns the indices of the 16 closest Gaussians for each point. After attaining the density, equation

$$f(p) = \pm s_{g^*} \sqrt{-2 \log(d(p))}$$

to calculate SDF value  $f(p)$ .

On the other hand, estimated “ideal” SDF values are derived by comparing the depth values of points in camera space with the depth values retrieved from a depth map. To begin with, the Gaussian coordinates are transformed from world coordinates to camera coordinates. Then, the depth of Gaussians in images is exactly the z-axis values in camera coordinates. Before retrieving the depth of Gaussians from the depth map, we take the 3D Gaussians in camera space and project them into a 2D image plane. After that, the 2D coordinates are normalized to fit the depth image size. When all things are ready, depth is obtained with the “grid\_sample” function provided by PyTorch, taking projected 2D locations as inputs, and sampling the depth values from the depth image on the corresponding position. Then, the “ideal” SDF values are the differences between two depth values, one from the z-axis in camera space, and the other from sampling from the depth map.

Finally, regularization terms are the averaged SDF differences between the Gaussian and “ideal” SDF values.

## Chapter 5

# Experiments

The codes for experiments are developed on diffusers library[29] and Nerfstudio[28], while the device is equipped with an NVIDIA RTX 3090 GPU, with ample system memory (45 GB), and installed PyTorch 2.1.2 with CUDA 11.8 for hardware-accelerated computations. Specifically, CUDA 11.8 is for gsplat[33] library used in Nerfstudio, and Python 3.10 is for PyTorch3D 0.7.7 [34]. The environment is set up following the guide provided by Nerfstudio[28]. Despite of the settings, Cecum data from Colonoscopy 3D Video Dataset (C3VD) is used.

In this chapter, we firstly declare the dataset we used for training and evaluation. Then, we will look through the experiments we conducted, from diffusion models to the Gaussian Splatting model, and to the novel methods we propose.

### 5.1 Dataset

Cecum colonoscopy videos are used for the experiments, which is provided by C3VD. The videos are at a resolution of 1350 x 1080 and of length from 276 to 465 frames. For each video, the frames are divided into training ,testing and validation sets in a similar ratio as experiments in [14], 6:2:2.

The diffusion models are pretrained on all cecum frames for 150 iterations, during which images are transformed to resolution of 256 x 256. For inference from noisy novel-view images in a specific case, they are further refined using single cecum video frames over an additional 100 iterations. Also, single cecum video frames are used for every 3D Gaussian model reconstruction. Automatically, the downscale factor is set to 2 in Nerfstudio, meaning downscale images by 4.

Images that deviate from original trajectory are not selected for test and evaluation. The reason is that we can not guarantee the diffused images are complete align with the real scene we aim to reconstruct. They can only serve as priors for novel view image synthesis and scene reconstruction. However, we still rendered several images away from original trajectory for comparison.

### 5.2 DDIM and DDPM Model

Two basic diffusion models are chosen: DDPM and DDIM, among which, DDPM is trained on the Markovian process, but DDIM on the non-Markovian process, depending on both original and last step inputs. we firstly trained the two models for 50 iterations on cecum.t1.a data attained from C3VD [8], so as to discover their properties.

Having chosen the suitable model-DDIM, we continue the training on the DDIM model with cecum.t2.a abd cecum.t4.a data, and for each dataset, we trained another 50 iterations. Finally,

considering that we will experiment with cecum\_t1\_a data for our novel methods, we again train the model with cecum\_t1\_a data for another 100 iterations to overfit the dataset. Then, we get the diffusion model we want.

### 5.3 Gaussian Splatting and instant-NGP

Gaussian Splatting model code is ready to use in Nerfstudio, so we train it for 10000 steps (defined by the command in Nerfstudio: “`–max-num-iterations`”) as a baseline for later comparison. Additionally, we re-train it for 7000 steps, the same as what is done for SuGaR[15] pretraining. This gives a model capable of rendering comparably reasonable images letting the diffusion model reasoning. Besides, to compare with baselines, we also trained Instant-NGP[35] for 10000 steps.

### 5.4 DDIM-Splatting and SDF-DDIM-Splatting

We conduct an ablation study on the proposed novel methods, via changing the probability of sampling novel view images,  $\lambda_{ddim}$  and  $\lambda_{sdf}$  the weight of their losses. Different performances among the settings give us insights into the introduction of the DDIM model.

Below, Table 5.1, is the table of the settings we have experimented with.

|       | $\lambda_{ddim}$ | $\lambda_{sdf}$ | $P_{novel\_view}$ |
|-------|------------------|-----------------|-------------------|
| Exp 1 | 0.2              | 0               | 0.1               |
| Exp 2 | 0.02             | 0               | 0.1               |
| Exp 3 | 0.002            | 0               | 0.1               |
| Exp 4 | 0.002            | 1e-3            | 0.1               |
| Exp 5 | 0.002            | 1e-4            | 0.1               |
| Exp 6 | 0.002            | 0               | 0.2               |
| Exp 7 | 0.002            | 0               | 0.01              |

*Table 5.1: Ablation Study Settings,*  
which will be grouped and analysed separately.

The experiments are grouped to conduct ablation study: Experiment 1-3 together with the original Gaussian Splatting method are viewed to explore the influence of weights on the losses where diffused images from novel views are taken as ground truth; Experiment 3 and 5 are compared together with the 3DGS method and Instant-NGP methods for method ablation study, investigating how different methods affect performance; Experiment 5-7 are compared with 3DGS as a baseline to evaluate the influence of the frequency of novel-view sampling on model performance; finally, Experiment 3-5 are designed to explore how SDF regularization will change performance.

## Chapter 6

# Results and Discussion

Through a series of experiments and evaluations, the proposed integration has demonstrated improvements in several key metrics, particularly PSNR, FPS, and LPIPS.

### 6.1 DDIM-Splatting Ablation

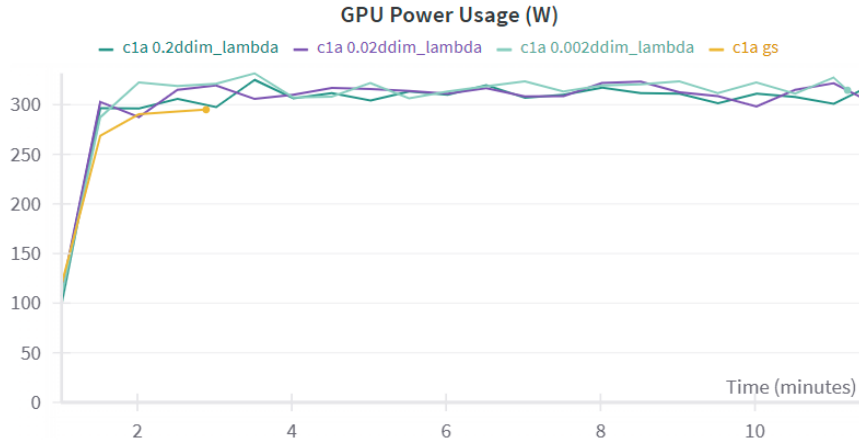


Figure 6.1: GPU Run-time, in terms of original Gaussian Splatting method, and new DDIM-Splatting methods with different weights for novel view training— $ddim\_lambda$

By introducing the DDIM model during training, we can train with novel view images, and take diffused images as ground truth.  $\lambda_{ddim}$  is introduced, which is a weight multiplied before the main loss when we train with novel view images. In this ablation study, we assign 0.2, 0.02, and 0.002 to  $\lambda_{ddim}$  and compare with the Gaussian Splatting[1] as a baseline.

From the GPU power usage chart, Figure 6.1, we observe that introducing the DDIM models increases the overall power consumption during training but does not significantly exceed the baseline consumption of traditional 3DGS, and so does the training time. The power usage stabilizes quickly and remains consistent.

In Figure 6.4 and 6.5, evaluation matrices are compared across DDIM-Splatting variations with 3DGS as baseline. By viewing the FPS values, while the DDIM integration adds computational complexity, it does not drastically impair the runtime efficiency, but boosts the speed of rendering,

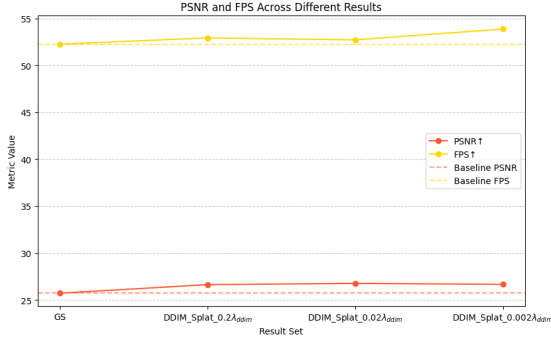


Figure 6.2: Chart 1:

PSNR and FPS values evaluating the influence of different weights- $\lambda_{ddim}$  for novel view training-on model inference performance. The higher PSNR and FPS values the better the models. On average, DDIM\_Splat models outperform original Gaussian Splatting(GS) methods considering PSNR and FPS values.

While PSNR for different  $\lambda_{ddim}$  are similar, FPS raises when we change the weight to 0.002.

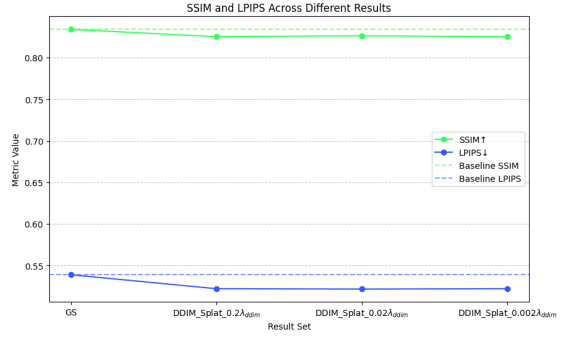


Figure 6.3: Chart 2:

SSIM and LPIPS values evaluating the influence of different weights- $\lambda_{ddim}$  for novel view training-on image synthesis quality. SSIM is the higher the better, but LPIPS is the opposite. Clearly, original GS are better in SSIM. However, DDIM\_Splat are better in LPIPS on average.

Figure 6.4: Ablation Study 1

-Same novel-view sampling probability(0.1), and no SDF regularization for DDIM-Splatting models. Demonstrate the influence of different weights- $\lambda_{ddim}$  (0.2, 0.02, 0.002) on performance.

|   | Extraction method                 | PSNR   | FPS    | SSIM  | LPIPS |
|---|-----------------------------------|--------|--------|-------|-------|
| 0 | GS                                | 25.723 | 52.248 | 0.834 | 0.539 |
| 1 | DDIM_Splat_0.2λ <sub>ddim</sub>   | 26.636 | 52.920 | 0.825 | 0.522 |
| 2 | DDIM_Splat_0.02λ <sub>ddim</sub>  | 26.769 | 52.705 | 0.826 | 0.522 |
| 3 | DDIM_Splat_0.002λ <sub>ddim</sub> | 26.666 | 53.867 | 0.825 | 0.522 |

Figure 6.5: Ablation Study 1 Table Comparison:

Same novel-view sampling probability(0.1), and no SDF regularization for DDIM-Splatting models. Models are better when PSNR, FPS, SSIM higher, and LPIPS lower.

making it a feasible approach for improving 3D reconstruction without excessive resource demands.

The PSNR results confirm that DDIM enhances image clarity, as demonstrated by the consistent increase across different DDIM configurations. Surprisingly, despite the common concern of diffusion models introducing noise, our experiments show that DDIM does not diminish the PSNR. Instead, it improves the perceptual quality of images, as evidenced by the lower LPIPS values. This indicates that DDIM not only constrains Gaussians in structure but also aligns better with human perception.

On the other hand, SSIM values showed a slight decline when DDIM was incorporated. This is likely due to the fixed, constrained view used for evaluation, being the same as what traditional 3DGS focuses on training with. Given that DDIM is designed to generate novel views, its lower SSIM score reflects its capability to deviate from the training trajectory, offering potential advantages in broader applications where novel view synthesis is required.

In summary, incorporating DDIM into the 3DGS pipeline yields improved performance in terms of PSNR, FPS, and LPIPS, with a minor trade-off in SSIM due to the nature of the evaluation. The increased GPU power consumption is manageable and does not compromise overall system

efficiency in rendering.

## 6.2 Multiple Methods Ablation

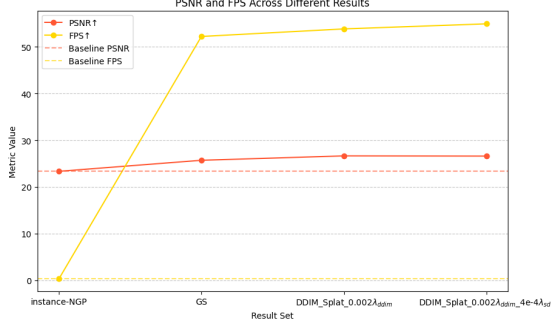


Figure 6.6: Chart 1:

PSNR and FPS values evaluating the influence of different methods on inference performance. Gaussian Splatting-based methods render much faster than NeRF based method—Instant-NGP, while also reduce the noise in rendered images.

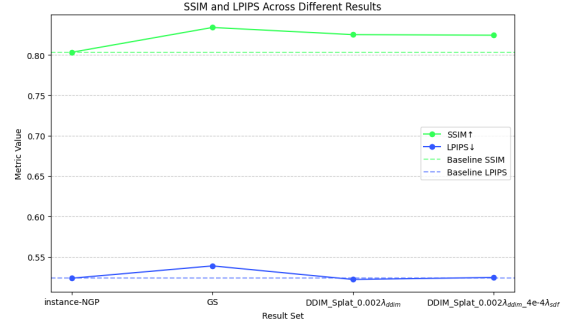


Figure 6.7: Chart 2:

SSIM and LPIPS values evaluating the influence of different methods on image synthesis quality. Taking Instant-NGP model as the baseline, SSIM and LPIPS are also better for GS based models, but GS has the highest SSIM value.

Figure 6.8: Ablation Study 2:

same novel-view sampling probability(0.1) for DDIM-Splatting models. Demonstrate the influence of different methods (Instant-NGP, GS, DDIM\_Splatt, and DDIM\_Splat\_SDF) on performance.

|   | Extraction method                | PSNR   | FPS    | SSIM  | LPIPS |
|---|----------------------------------|--------|--------|-------|-------|
| 0 | instance-NGP                     | 23.359 | 0.311  | 0.803 | 0.523 |
| 1 | GS                               | 25.723 | 52.248 | 0.834 | 0.539 |
| 2 | DDIM_Splat_0.002λ_ddim           | 26.666 | 53.867 | 0.825 | 0.522 |
| 3 | DDIM_Splat_0.002λ_ddim_4e-4λ_sdf | 26.622 | 54.951 | 0.824 | 0.524 |

Figure 6.9: Ablation Study 2 Table Comparison:

same novel-view sampling probability(0.1) for DDIM-Splatting models

In the second ablation study, we explore how different methods influence performance. Demonstrated in Figure 6.8 and 6.9, the instance-NGP method, while maintaining a reasonable score in terms of rendered image quality (for example, lower LPIPS than the Gaussian Splatting method), exhibits significant limitations in speed. Despite being one of the fastest NeRF-based methods, it lags far behind Gaussian-based explicit rendering approaches.

The introduction of DDIM into the Gaussian Splatting framework clearly outperforms the traditional GS method. DDIM-Splatting achieved the highest PSNR and lowest LPIPS, confirming its superiority in rendering clarity and perceptual quality. Furthermore, the integration of DDIM-Splatting with SDF regularization led to the highest FPS, reflecting better efficiency in computational performance.

As demonstrated in Figure 6.10, the images effectively illustrate the influence of varying  $\lambda$  values and the integration of SDF on the resulting image quality. These images were rendered from a novel view, with the ground truth generated by rendering from a novel perspective followed by

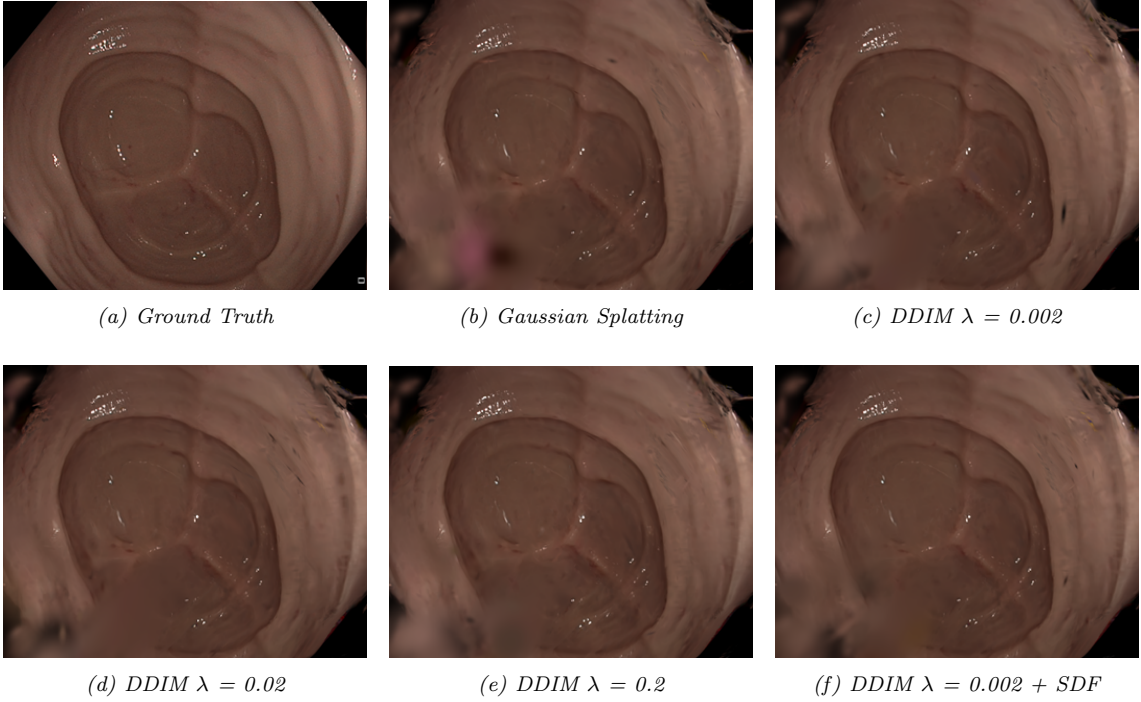


Figure 6.10: Renders:

same novel-view sampling probability(0.1) for DDIM-Splatting models. The Ground truth image is the novel view image rendered by 3DGS and diffused by DDIM

refinement using the DDIM process. DDIM-Splatting models provide more detailed information in previously vague regions rendered by 3DGS. Particularly, the one using SDF regularization provides better reconstructions than any others, despite the lower scores the model gets in Figure 6.9.

Although the current performance is promising, especially with the highest PSNR and lowest LPIPS, further enhancements could be achieved by incorporating more novel view sampling during training. This will perhaps enable the model to generate more accurate reconstructions from previously unseen viewpoints, further optimizing its performance in real-world applications. Therefore, we proceed with additional experiments, exploring various probabilities of sampling novel view images to be used as training data.

### 6.3 Novel-view Sampling Ablation

In this ablation study, we investigated the effect of introducing novel view sampling at different probabilities on the performance of the DDIM-Splatting model. As illustrated in Figure 6.13 and Figure 6.14, it turns out that the model using a 0.01 probability of sampling novel views achieves the best overall performance, with the highest PSNR and lowest LPIPS, indicating a superior performance on image clarity and perceptual quality. The model with a 0.1 probability of sampling novel views shows similar PSNR performance but has a faster rendering speed, and higher FPS, which is crucial for real-time applications.

In comparison to the baseline Gaussian Splatting method, all models incorporating novel views outperform the 3DGS model across most metrics, with lower LPIPS and similar or higher PSNR values. This indicates that novel view sampling introduces more diverse perspectives, improving the model’s ability to generalize across different views.

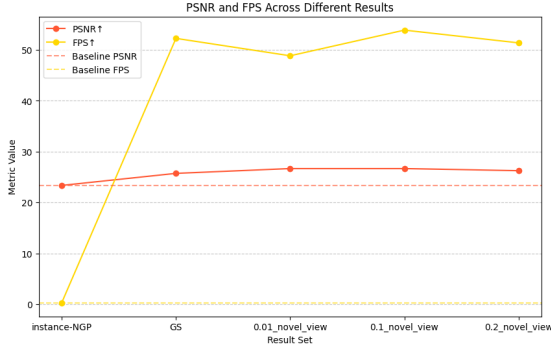


Figure 6.11: Chart 1:

PSNR and FPS values evaluating the influence of different probability of sampling novel views for training on model inference performance. NeRF-based Instant-NGP renders the slowest in evident contrast with GS-based methods. The model with 0.01 probability of using novel view for training is slower in rendering than other DDIM-Splatting models but have a slightly higher PSNR value.

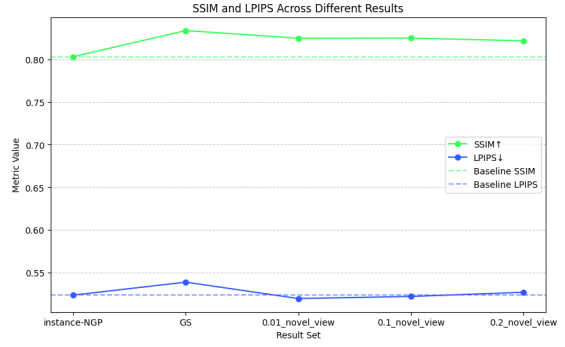


Figure 6.12: Chart 2:

SSIM and LPIPS values evaluating the influence of different probability of sampling novel views for training on image synthesis quality. Still, GS method have the highest SSIM value, but also highest LPIPS, which means the model does not perform well under LPIPS evaluation.

Figure 6.13: Ablation Study 3:

same diffusion lambda (0.002), and no SDF regularization for DDIM-Splatting models. Demonstrate the influence of probability of sampling novel views for training on model performance

|   | Extraction method | PSNR   | FPS    | SSIM  | LPIPS |
|---|-------------------|--------|--------|-------|-------|
| 0 | instance-NGP      | 23.359 | 0.311  | 0.803 | 0.523 |
| 1 | GS                | 25.723 | 52.248 | 0.834 | 0.539 |
| 2 | 0.01_novel_view   | 26.667 | 48.826 | 0.825 | 0.519 |
| 3 | 0.1_novel_view    | 26.666 | 53.867 | 0.825 | 0.522 |
| 4 | 0.2_novel_view    | 26.252 | 51.365 | 0.822 | 0.527 |

Figure 6.14: Ablation Study 3 Table Comparison:

same diffusion lambda (0.002), and no SDF regularization for DDIM-Splatting models

The slight underperformance of the model with a 0.2 probability of novel views in both PSNR and SSIM could be attributed to the limited pre-training steps (7000 steps), which may not have been sufficient for the model to fully refine its understanding of novel views. While too much focus on novel views might introduce noise, a moderate amount, as evidenced by the 0.01 and 0.1 probability models, can enhance performance significantly.

## 6.4 SDF Regularization Level Ablation

Knowing from Section 6.2, the method integrating SDF regularization can only render images with a little higher quality than the Instant-NGP method, with the next-to-last SSIM, and LPIPS scores. However, it renders the fastest among them. Therefore, we conduct this ablation study, via varying the level of SDF regularization. From Figure 6.17 and Figure 6.18, the rendering quality indeed declines as we assign more weight to the regularization. We observe that PSNR is

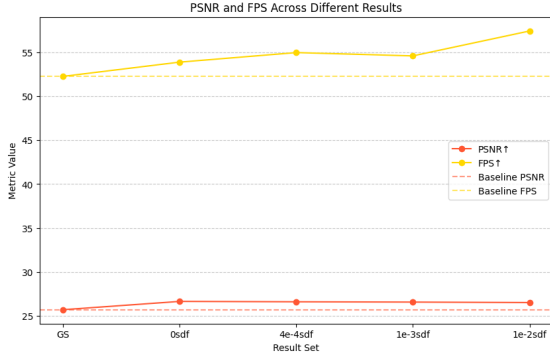


Figure 6.15: Chart 1:

PSNR and FPS values evaluating the influence of different weights of SDF regularization on model inference performance. It seems more weights on SDF regularization result in a faster rendering in general, despite the slight drop of FPS for the model with 1e-3 SDF weight. Also, regardless of SDF weights, adopting DDIM-Splatting method generally make the rendered image less noise.

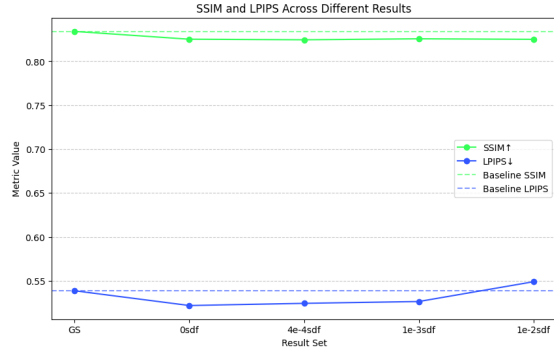


Figure 6.16: Chart 2:

SSIM and LPIPS values evaluating the influence of different weights of SDF regularization on image synthesis quality. Still, GS method have the highest SSIM value, but not the highest LPIPS in this group of study. The model with 1e-2 SDF weight have the highest LPIPS value, which means large weight on SDF regularization could harm the performance under LPIPS evaluation.

Figure 6.17: Ablation Study 4:

same diffusion lambda (0.002) and novel view sampling probability (0.1) for DDIM-Splatting models. Demonstrate the influence of weights of SDF regularization on model performance.

|   | Extraction method | PSNR   | FPS    | SSIM  | LPIPS |
|---|-------------------|--------|--------|-------|-------|
| 0 | GS                | 25.723 | 52.248 | 0.834 | 0.539 |
| 1 | 0sdf              | 26.769 | 52.705 | 0.826 | 0.522 |
| 2 | 4e-4sdf           | 26.622 | 54.951 | 0.824 | 0.524 |
| 3 | 1e-3sdf           | 26.591 | 54.577 | 0.826 | 0.526 |
| 4 | 1e-2sdf           | 26.539 | 57.408 | 0.825 | 0.549 |

Figure 6.18: Ablation Study 4 Table Comparison:

same diffusion lambda (0.002) and novel view sampling probability (0.1) for DDIM-Splatting models

diminished and LPIPS gradually increases, while SSIM values are relatively stable for all models with DDIM integrated.

Although the methods integrating SDF regularization do not surpass others in terms of image quality, and the image quality deteriorates as the weight of SDF regularization increases, another interesting observation has been made. As demonstrated in Figure 6.19 and Figure 6.20, by involving SDF regularization, the number of Gaussians is dropped drastically, resulting in rendering in much higher speeds, while still preserving reasonable image rendering quality. It suggests that the models can take advantage of SDF regularization in real-time rendering, so as to support surgical applications that have high demand for rendering speeds, and meanwhile integrate DDIM models to guarantee the image quality: despite the compromised scores with increasing SDF regularization weights, the performance still better than original Gaussian Splatting.

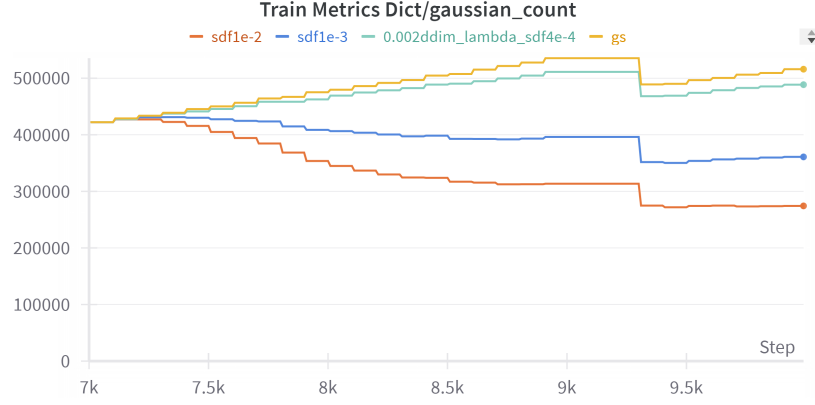


Figure 6.19: Gaussian count:

same diffusion lambda (0.002) and novel view sampling probability (0.1) for DDIM-Splatting models. Gaussian counts decrease with the increment of weights of SDF regularization, which means SDF regularization effectively constrains the Gaussian number making the models simpler.

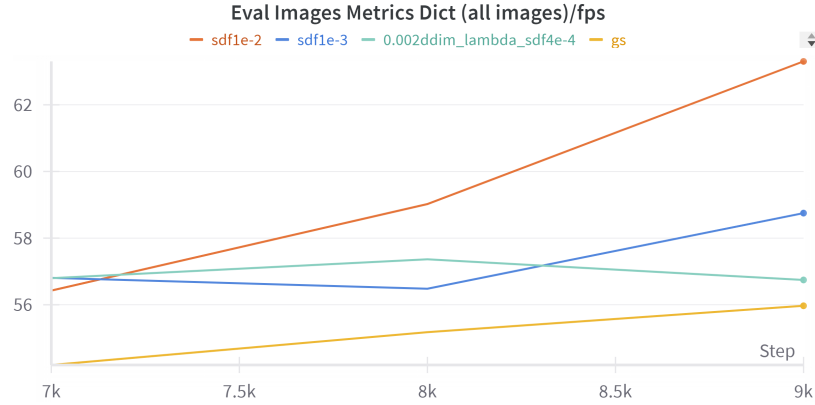


Figure 6.20: Frames per Second:

same diffusion lambda (0.002) and novel view sampling probability (0.1) for DDIM-Splatting models. The evaluation over all images is conducted every 1000 steps. Higher values indicate faster rendering. The model with the weight of SDF regularization of  $1e-2$  rendered fastest. With the increment of the weights, rendering speeds after 3000 more training also increase.

## Chapter 7

# Conclusion and Future Work

This project reviews the novel 3D scene reconstruction techniques, focusing NeRF, and 3D Gaussian Splatting. Additionally, in light of the unique challenges of endoscopy 3D reconstruction, such as limited camera angles and datasets, rigorous demand over generated data consistency and anatomical accuracy, this work introduced the Denoising Diffusion Implicit Model (DDIM) into the colon 3D reconstruction pipeline. The primary objective is to strike a balance between data augmentation and the preservation of original features, thereby improving the model without compromising the precision and reliability required by medical standards. Moreover, to further constrain the distribution of Gaussians onto the reconstructed organ’s surface, Signed Distance Function (SDF) regularization was also incorporated. Our experimental results confirm that the integration of DDIM into 3DGS leads to improved performance.

However, there remain areas for further investigation, including the optimization of hyperparameters (such as novel view sampling rates, loss function weights, and training steps, among others) and the simplification of the codebase to ensure efficient use of computational resources. Additionally, during training how to select novel views deviating from the captured trajectories matters to the training efficiency, as images rendered from certain views after pretraining can be already clear with little loss or very vague with little information. Moreover, since we incorporate interaction with the model in real-time during training, reinforcement learning techniques are probably applicable to further improve the proposed DDIM-Splatting model. These refinements are essential to fully realize the potential of this approach in clinical applications.

# Bibliography

- [1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3d gaussian splatting for real-time radiance field rendering,” 2023.
- [2] J. L. Schonberger and J.-M. Frahm, “Structure-from-motion revisited,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 4104–4113, 2016.
- [3] M. Zwicker, H. Pfister, J. Van Baar, and M. Gross, “Ewa volume splatting,” in *Proceedings Visualization, 2001. VIS’01.*, pp. 29–538, IEEE, 2001.
- [4] Wikipedia contributors, “Spherical harmonics — Wikipedia, the free encyclopedia,” 2024. [Online; accessed 7-August-2024].
- [5] Z. Yang, J. Dai, and J. Pan, “3d reconstruction from endoscopy images: A survey,” *Computers in Biology and Medicine*, vol. 175, p. 108546, 2024.
- [6] K. Cheng, Y. Ma, B. Sun, Y. Li, and X. Chen, “Depth estimation for colonoscopy images with self-supervised learning from videos,” in *Medical Image Computing and Computer Assisted Intervention – MICCAI 2021* (M. de Bruijne, P. C. Cattin, S. Cotin, N. Padoy, S. Speidel, Y. Zheng, and C. Essert, eds.), (Cham), pp. 119–128, Springer International Publishing, 2021.
- [7] A. Rau, S. Bano, Y. Jin, P. Azagra, J. Morlana, R. Kader, E. Sanderson, B. J. Matuszewski, J. Y. Lee, D.-J. Lee, E. Posner, N. Frank, V. Elangovan, S. Raviteja, Z. Li, J. Liu, S. Lalithkumar, M. Islam, H. Ren, L. B. Lovat, J. M. Montiel, and D. Stoyanov, “Simcol3d — 3d reconstruction during colonoscopy challenge,” *Medical Image Analysis*, vol. 96, p. 103195, 2024.
- [8] T. L. Bobrow, M. Golhar, R. Vijayan, V. S. Akshintala, J. R. Garcia, and N. J. Durr, “Colonoscopy 3d video dataset with paired depth from 2d-3d registration,” *Medical Image Analysis*, vol. 90, p. 102956, Dec. 2023.
- [9] R. Wu, B. Mildenhall, P. Henzler, K. Park, R. Gao, D. Watson, P. P. Srinivasan, D. Verbin, J. T. Barron, B. Poole, and A. Holynski, “Reconfusion: 3d reconstruction with diffusion priors,” 2023.
- [10] A. Molaei, A. Aminimehr, A. Tavakoli, A. Kazerouni, B. Azad, R. Azad, and D. Merhof, “Implicit neural representation in medical imaging: A comparative survey,” 2023.
- [11] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, “Nerf: Representing scenes as neural radiance fields for view synthesis,” 2020.
- [12] Y. Wang, Y. Long, S. H. Fan, and Q. Dou, “Neural rendering for stereo 3d reconstruction of deformable tissues in robotic surgery,” 2022.

- [13] R. Zha, X. Cheng, H. Li, M. Harandi, and Z. Ge, “Endosurf: Neural surface reconstruction of deformable tissues with stereo endoscope videos,” 2023.
- [14] B. Huang, Y. Wang, A. Nguyen, D. Elson, F. Vasconcelos, and D. Stoyanov, “High-fidelity endoscopic image synthesis by utilizing depth-guided neural surfaces,” 2024.
- [15] A. Guédon and V. Lepetit, “Sugar: Surface-aligned gaussian splatting for efficient 3d mesh reconstruction and high-quality mesh rendering,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 5354–5363, 2024.
- [16] M. Yu, T. Lu, L. Xu, L. Jiang, Y. Xiangli, and B. Dai, “Gsdf: 3dgs meets sdf for improved rendering and reconstruction,” *arXiv preprint arXiv:2403.16964*, 2024.
- [17] L. Zhu, Z. Wang, Z. Jin, G. Lin, and L. Yu, “Deformable endoscopic tissues reconstruction with gaussian splatting,” *arXiv preprint arXiv:2401.11535*, 2024.
- [18] S. Bonilla, S. Zhang, D. Psychogyios, D. Stoyanov, F. Vasconcelos, and S. Bano, “Gaussian pancakes: Geometrically-regularized 3d gaussian splatting for realistic endoscopic reconstruction,” *arXiv preprint arXiv:2404.06128*, 2024.
- [19] R. Ma, R. Wang, Y. Zhang, S. Pizer, S. K. McGill, J. Rosenman, and J.-M. Frahm, “Rnnslam: Reconstructing the 3d colon to visualize missing regions during a colonoscopy,” *Medical image analysis*, vol. 72, p. 102100, 2021.
- [20] Z. Yang, K. Chen, Y. Long, and Q. Dou, “Efficient data-driven scene simulation using robotic surgery videos via physics-embedded 3d gaussians,” *arXiv preprint arXiv:2405.00956*, 2024.
- [21] X. Li, H. Wang, and K.-K. Tseng, “Gaussiandiffusion: 3d gaussian splatting for denoising diffusion probabilistic models with structured noise,” 2023.
- [22] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial networks,” 2014.
- [23] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” *Advances in neural information processing systems*, vol. 33, pp. 6840–6851, 2020.
- [24] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” 2022.
- [25] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” 2022.
- [26] H. Chen, J. Gu, A. Chen, W. Tian, Z. Tu, L. Liu, and H. Su, “Single-stage diffusion nerf: A unified approach to 3d generation and reconstruction,” in *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 2416–2425, 2023.
- [27] H. Wang, X. Du, J. Li, R. A. Yeh, and G. Shakhnarovich, “Score jacobian chaining: Lifting pretrained 2d diffusion models for 3d generation,” 2022.
- [28] M. Tancik, E. Weber, E. Ng, R. Li, B. Yi, J. Kerr, T. Wang, A. Kristoffersen, J. Austin, K. Salahi, A. Ahuja, D. McAllister, and A. Kanazawa, “Nerfstudio: A modular framework for neural radiance field development,” in *ACM SIGGRAPH 2023 Conference Proceedings*, SIGGRAPH ’23, 2023.

- [29] P. von Platen, S. Patil, A. Lozhkov, P. Cuenca, N. Lambert, K. Rasul, M. Davaadorj, D. Nair, S. Paul, W. Berman, Y. Xu, S. Liu, and T. Wolf, “Diffusers: State-of-the-art diffusion models.” <https://github.com/huggingface/diffusers>, 2022.
- [30] M. S. Nixon and A. S. Aguado, “4 - low-level feature extraction (including edge detection),” in *Feature Extraction and Image Processing for Computer Vision (Fourth Edition)* (M. S. Nixon and A. S. Aguado, eds.), pp. 141–222, Academic Press, fourth edition ed., 2020.
- [31] W. Yifan, F. Serena, S. Wu, C. Öztireli, and O. Sorkine-Hornung, “Differentiable surface splatting for point-based geometry processing,” *ACM Transactions on Graphics*, vol. 38, p. 1–14, Nov. 2019.
- [32] J. L. Schönberger, E. Zheng, M. Pollefeys, and J.-M. Frahm, “Pixelwise view selection for unstructured multi-view stereo,” in *European Conference on Computer Vision (ECCV)*, 2016.
- [33] V. Ye and A. Kanazawa, “Mathematical supplement for the `gsplat` library,” 2023.
- [34] N. Ravi, J. Reizenstein, D. Novotny, T. Gordon, W.-Y. Lo, J. Johnson, and G. Gkioxari, “Accelerating 3d deep learning with pytorch3d,” *arXiv:2007.08501*, 2020.
- [35] T. Müller, A. Evans, C. Schied, and A. Keller, “Instant neural graphics primitives with a multiresolution hash encoding,” *ACM Trans. Graph.*, vol. 41, pp. 102:1–102:15, July 2022.